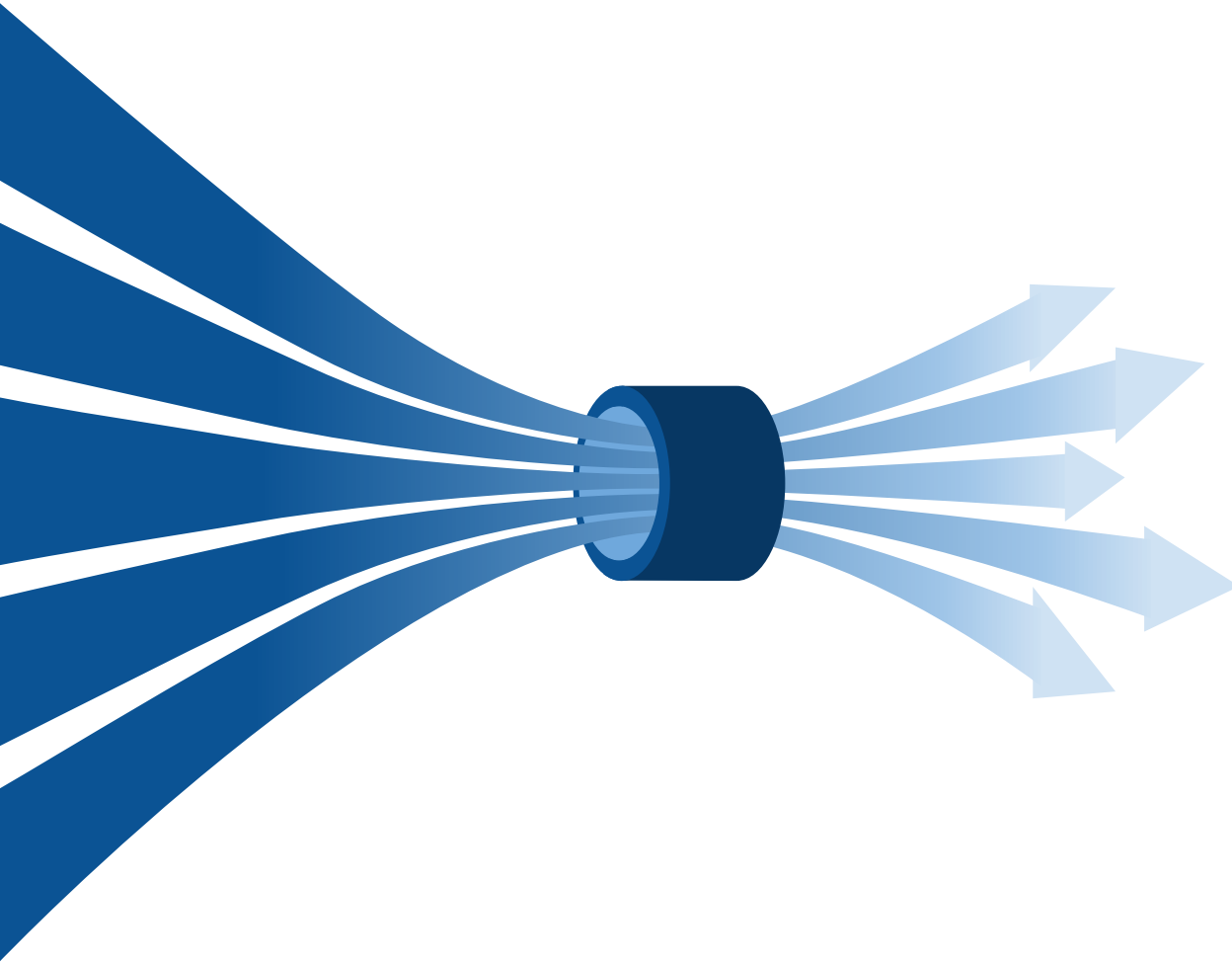




Session 4

The apply family

lapply

`lapply` takes a vector or list `x`, and applies the function `FUN` to each of its members. If `FUN` requires additional arguments, you pass them after you've specified `x` and `FUN ( . . . )`. The output of `lapply()` is a list, the same length as `x`, where each element is the result of applying `FUN` on the corresponding element of `x`.

```
lapply(X, FUN, ...)
```

lapply and anonymous functions

Previously, you learned that functions in R are objects in their own right. This means that they aren't automatically bound to a name. When you create a function, you can use the assignment operator to give the function a name. It's perfectly possible, however, to not give the function a name. This is called an anonymous function:

```
# Named function
triple <- function(x) { 3 * x }

# Anonymous function with same implementation
function(x) { 3 * x }

# Use anonymous function inside lapply()
lapply(list(1,2,3), function(x) { 3 * x })
```

Use lapply with additional arguments

`lapply()` provides a way to handle functions that require more than one argument, such as the `multiply()` function:

```
multiply <- function(x, factor) {  
  x * factor  
}  
lapply(list(1,2,3), multiply, factor = 3)
```

sapply

You can use `sapply()` similar to how you used `lapply()`. The first argument of `sapply()` is the list or vector `x` over which you want to apply a function, `FUN`. Potential additional arguments to this function are specified afterwards `(...)`:

```
sapply(X, FUN, ...)
```

sapply with function returning vector

In the previous exercises, you've seen how `sapply()` simplifies the list that `lapply()` would return by turning it into a vector. But what if the function you're applying over a list or a vector returns a vector of length greater than 1?